

Problem B:

Regression Failure Bucketing

Grant Gung-Yu Pan
(Synopsys, Inc.)

Q&A

Q1. I am writing to inquire about the LLM API configurations described in the problem statement.

According to Section 3.4 (LLM Usage & Configuration), the final evaluation will be conducted using two specific endpoints: one for a Qwen model and another for GPT-4o. While the document mentions that contestants may use their own endpoints during the development phase, we would like to know if the organizers provide any official test API endpoints.

Could you please clarify if such testing resources are available to contestants before the alpha submission?

A1. No, we cannot provide LLM endpoint outside Synopsys.

This is the nature of GenAI applications in EDA industry.

Build application with development endpoint, ship to customer, and use with deployment endpoint.

Once the model is the same, the behavior should be the same.

Q2. I am working on ICCAD Problem B: Regression Failure Bucketing, and I would like to clarify the expected granularity of the injected bugs/faults.

From the problem statement, each golden bucket corresponds to one injected bug, and each failure case is assumed to be primarily caused by one injected bug. However, the statement does not specify the granularity or scope of these injected bugs.

Could you clarify whether the injected bugs are limited to certain coarse-grained functional categories or RTL subsystems, such as decoder, debug controller, CSR/privilege logic, branch/PC control, load-store unit, memory interface, etc.? Or should we assume that the injected bugs may occur anywhere in the modified Ibex RTL without a predefined fault taxonomy?

In other words, is it reasonable for contestants to build a diagnosis-inspired feature extractor that maps failure symptoms to coarse suspect regions of the Ibex design, or are the injected bugs intended to be arbitrary RTL modifications where no such coarse category assumption should be made?

I understand that the actual injected bugs and golden buckets cannot be disclosed. I am only asking whether there is any intended coarse-grained fault scope or fault model that participants may rely on when designing the bucketing algorithm.

A2. There is no predefined fault taxonomy or fixed list of coarse subsystems, and different bugs can produce overlapping symptoms across coarse-grained subsystems. In short, feel free to exploit RTL/DV knowledge for feature engineering, but please design the bucketing algorithm to be open-ended over the bug space rather than assume a fixed fault model.

Q3. I would like to ask which version of the Ibex core was used to generate the benchmark cases for this contest. If possible, could you provide the specific tag, branch, or commit hash of the Ibex repository?

A3. You can check out using the hash from the upstream lowRISC Ibex repo:
git clone <https://github.com/lowRISC/ibex.git>
git checkout 8ce399dbe678f0a66856ac302ec7609ba366d8fd
Note: the RTL was locally modified for bug injection on top of this commit.

Q4. I am working on ICCAD Problem B: Regression Failure Bucketing, and I would like to ask whether the official evaluation environment will provide GPU access for submitted programs. If GPU access is available, could you please let us know the expected GPU model and memory size?

A4. The host machine is not equipped with GPU. If your program uses LLM, it connects to the endpoint with GPU via HTTP.

Q5. A few days ago, you updated the golden files in the test cases. We noticed that in Benchmark 1, the three log files in Case 5 and Case 9 are almost identical, with the only difference being their runtime timestamps. However, in the official golden files,

they are assigned to different buckets. This does not seem to be a sufficient basis for placing them into different buckets. Therefore, we are writing to ask whether there might be an error in the golden files.

A5. Thank you for the careful investigation. The same syndrome you see is actually coming from two different bugs, while failing on the same regression test. Although this is very common in industrial regression bucketing, to reduce the noise from development and evaluation, we have excluded all these conflicting cases from the benchmark sets (and made some swapping for size reasons), both public and private. Please refer to the update benchmark sets.

Q6. We would like to ask about the evaluation/testing environment for the contest, since the runtime limit is important for our implementation. Could you please provide some details about the machine used for evaluation, such as the CPU model, the maximum number of CPU cores/threads we are allowed to use, and the available memory size?

A6. The CPU has 32 logical cores available to your process with ~128GB RAM. For Alpha/Beta tests, we will enlarge the timeout threshold by 3X/2X and report the actual runtime, so you can optimize your program accordingly.

Q7. Does the modified source code belong to the CPU design or the simulation tool? In other words, are we trying to detect a bug in the design or in the tool? Does the assembly language in the trace file represent the execution process of the simulation tool or the CPU design?

A7. The bugs are injected into the RTL source code where the assembly trace is dumped from simulation of the CPU design. The simulator (VCS) and testbench (UVM) are oracle.

Q8. Could you please let us know which ISS version you are using? Also, is the UVM version based on the Ibx version?

A8. Ibex uses Spike as a golden reference. UVM version is determined by the simulator where you can find in `sim.log.gz`: UVM-1.2

Q9. According to the latest announcement, teams working on Problems B should submit their files through the provided Google Drive links by placing them into the `alpha_test_submission` folder. In addition, the problem description states that the submitted program should be executed as follows:

```
regn_fail_bucketing -input -output -k
```

It also mentions that for programs written in Python, it is strongly recommended to pack the entire environment using PyInstaller, and that uploading the source code is optional. Only if the executable program cannot run, the evaluator will try running the source code instead.

We would like to confirm how the runtime environment and executable compatibility will be handled during evaluation. Since a PyInstaller executable is usually not guaranteed to run on every machine, it may still depend on the operating system, CPU architecture, glibc version, and native libraries used by Python packages.

In particular, we would like to ask the following questions:

1. What is the official evaluation environment for Problems B and C?
 - What operating system and version will be used?
 - What CPU architecture will be used?
 - Is there any information about the glibc version or Linux distribution?
2. For Python submissions using PyInstaller:
 - Should the executable be built on a specific Linux version to ensure compatibility?
 - Is the expected executable name exactly `regn_fail_bucketing`?
 - Should we submit only the PyInstaller executable, or should we also include the source code and dependency files as a fallback?
 - If our Python program uses packages such as NumPy, pandas, scikit-learn, or other packages with native libraries, is PyInstaller packaging sufficient?
 - If the executable fails due to environment compatibility, will the evaluator automatically try to run the submitted source code?
3. For Python source-code fallback:
 - What Python version will be available in the evaluation environment?

- Are we allowed to submit requirements.txt, environment.yml, or similar dependency files?
- Will the evaluator install the required Python packages before running the source code?
- Is internet access available during evaluation, or should all dependencies be included in the submission?

4. For C/C++ submissions:

- Should we submit source code with a Makefile/CMakeLists.txt, or should we submit a precompiled executable?
- What compiler and version will be used, such as GCC or Clang?
- Which C++ standard versions are allowed, such as C++17 or C++20?
- If we submit a precompiled executable, how should we avoid library compatibility issues such as glibc or libstdc++ version mismatches?

5. For Docker-based submissions:

- Is Docker allowed for Problems B and C?
- If Docker is allowed, should we submit a Dockerfile, a Docker image archive, or only build scripts?
- Are there any restrictions on Docker image size, base image, runtime permissions, or network access?

6. Before the deadline, is there any recommended way to verify that our submitted executable, source code, dependencies, file structure, and execution command are acceptable in the official evaluation environment?

Since late submissions and re-submissions due to file naming or submission errors will not be accepted, we would like to make sure that our submission can be correctly built and executed by the evaluation system.

A9. Thanks for the detailed questions. Build your executable on RHEL 8 / AlmaLinux 8 / CentOS 8 (glibc 2.28), targeting x86_64. Executable name must be exactly `regr_fail_bucketing`, as described in the released document. Third party packages (numpy, pandas, sklearn) are fine when bundled inside the PyInstaller binary. Submissions must be self-contained; do not rely on requirements.txt / pip installs / environment.yml since there is no internet access during eval. Bundle everything into the executable, either python or C++. Source (python) is still accepted only as a

fallback if the executable cannot start, but the source path must also be self-contained (refer to the example programs). For C++, submit a statically-linked (or RHEL 8-built) x86_64 executable. Docker is not used.

Q10. I am participating in Problem B.

I would like to clarify the allowed packaging format and evaluation environment for the Alpha Test submission.

According to the Problem B statement, Python programs are recommended to be packed using PyInstaller, and source code is optional.

Since Problem B submissions are submitted via Google Drive, could you please clarify:

1. What is the OS, CPU architecture, Python version, and glibc version of the Problem B evaluation environment?
2. If we submit a PyInstaller-packed executable, should it be built for x86_64 Linux?
3. Are external Python packages such as numpy, scipy, pandas, and scikit-learn allowed if they are bundled inside the executable?
4. If a submitted executable cannot run, will the evaluator try to run the provided source code? If so, are third-party packages installed in the evaluation environment, or should the submission be self-contained?

A10. Build your executable on RHEL 8 / AlmaLinux 8 / CentOS 8 (glibc 2.28), targeting x86_64. Third party packages (numpy, pandas, sklearn) are fine when bundled inside the PyInstaller binary. Bundle everything into the executable. Source is still accepted only as a fallback if the executable cannot start, but the source path must also be self-contained.

Q11. Exact Meaning of the "Buckets" Column: In

the benchmark table (e.g., Benchmark 4 lists "16" Buckets, Benchmark 6 lists "64" Buckets), does this number represent the exact amount of injected bugs (Golden Buckets) for that specific dataset?

A11. The number of buckets is equal to the number of bugs.

Q12. Deviation of the "Soft K" Parameter: The problem statement mentions that the given `--k` parameter serves as a "soft hint." During the hidden evaluations, will the value of `--k` passed to our program significantly deviate from the actual number of Golden Buckets, or should we expect only a minor variance from the true count?

A12. During evaluation, `--k` is passed as stated in [Section 3.2. Benchmark](#) for each benchmark set. What "soft" is the balanced accuracy stated in [Section 3.1. Figure-of-Merit](#) that even the resultant bucket number is not exact `k`, the solution can still get fair scores.

Q13. The problem statement specifies that the benchmark is based on Ibex commit: `8ce399dbe678f0a66856ac302ec7609ba366d8fd`

In this version, the function `compare_trace_csv()` is defined in:

``ibex/vendor/google_riscv-dv/scripts/instr_trace_compare.py`` , and is called from: ``ibex/dv/uvm/sim.py``

In the upstream source code, the default value of ``mismatch_print_limit`` is 5.

Therefore, unless this option is overridden, the regression report would normally print detailed information for up to five mismatch entries. In some cases, the comparator may also print a message such as: ``<number> instructions left in trace spike`` when unmatched architectural updates remain in the Spike trace.

However, We noticed that all officially provided non-UVM failure cases contain only a single `Mismatch[1]` block in `regr.log`.

Could you please clarify whether the benchmark generation flow included any of the following modifications?

1. Was `mismatch_print_limit` changed from 5 to 1?
2. Was `compare_trace_csv()` locally modified?
3. Were the generated regression logs post-processed or truncated so that only the first mismatch block was retained?

A13. Although the original benchmarking environment is cloned based on the Ibex commit, we have made several changes over years. Instead of relying on the modification history (transparent white box), please treat it as a black box with limited hints for this problem. The design of Problem B is to follow the common in practice that we cannot know exactly how data is produced; the data-driven clustering algorithm needs to be tolerant for some minor modifications.

Q14. This one also regarding the trace indices printed by ``instr_trace_compare.py``. In the upstream implementation, both the Ibex line and the Spike line in a normal mismatch block appear to use the same variable, ``trace_2_index - 1``, for the displayed index.

For example, the implementation seems to produce:

```
...
```

```
Mismatch[2]:
```

```
ibex[60] : ...
```

```
spike[60] : ...
```

```
...
```

even though `trace_1_index` and `trace_2_index` are maintained separately. In principle, a comparison such as the following could occur if the two trace streams advance differently:

```
...
```

```
Mismatch[2]:
```

```
ibex[60] : ...
```

```
spike[90] : ...
```

```
...
```

Could you please confirm whether the identical displayed indices are intentional, or whether this is only a logging issue in the upstream comparator? Also, was this part of the comparator modified when generating the benchmark logs?

A14. Similar to the above issue, please avoid assuming a perfect upstream comparator or post-processing.

Q15. Could you please confirm whether the following file was used without local modification? ``ibex/dv/uvm/riscv_dv_extension/testlist.yaml``

The test definitions, simulation options, ``compare_opts``, ``no_post_compare`` settings, and UVM test mappings in this file may affect feature extraction for UVM-related failures. Therefore, it would be helpful to know whether contestants may rely on the upstream version of this file at the specified commit.

A15. I have noticed the testlist is not exactly the same as the commit; it was referenced using another commit and might be modified locally. Again, in practice, the testlist is another stream that varies day to day. Please avoid the dependency on testlist.

Q16. Will the Beta Test use the same dataset as the Alpha Test, or will it use a different dataset that follows the same problem specifications?

Additionally, will the test cases be updated or expanded in the future?

A16. We will use the same 10 benchmark sets for Alpha, Beta, and Final Testing.

Q17. We noticed that the seed used in the logs for the public tests is always fixed at 7. We would like to ask whether this is also fixed in the hidden test cases, or whether it may be randomized.

A17. Seed can be random.

Q18. Would it be possible to share the individual scores for each of the 10 benchmark sets? This would help us better understand which sets performed well and which ones need improvement.

A18. Will share in Beta testing result.

Q19. Section 3.4 of the problem specification describes the LLM completion and embedding endpoints, but we could not find any published concurrency or rate-limit information. Could you please clarify the following for the official evaluation endpoints?

1. What is the maximum number of concurrent in-flight API requests allowed per submitted program/process for the completion and embedding endpoints?
2. Are there any RPS, RPM, TPM, total-request, or other quota limits?
3. Do the completion and embedding endpoints share the same concurrency/rate quota, or are their limits separate?
4. May contestants issue asynchronous requests in parallel across failure cases? If so, is there a recommended safe concurrency cap?
5. Should contestants expect HTTP 429 or similar throttling responses, and is there any recommended retry behavior?

We understand that network latency is included in the runtime and that excessive API calls causing a timeout result in a zero score. We would like to ensure that our

implementation does not overload the evaluation endpoint or rely on an unsupported concurrency level.

A19. This problem is designed to mimic real regression bucketing systems in industry; the algorithm needs to be resilient to the indicated LLM constraints. The completion and embedding share the same constraint, configured by IT, even not visible to the Topic Chair. 429-Too-Many-Requests is a common error, where a smart retry and a lightweight fallback are useful mechanisms to deal with. Although token consumption is not taken as cost in Problem B, bulk parallel LLM calls will trigger Rate Limit while the cooldown time is counted.

Q20. According to Section 3.2 of the problem statement, the ten benchmarks have different maximum runtime limits: 30 seconds, 100 seconds, or 300 seconds, depending on the benchmark scale. However, the program interface described in Section 3.3 only provides the following arguments:

```
regr_fail_bucketing --input <input.csv> --output <output.csv> --k <k>
```

It does not appear to include an explicit benchmark ID, benchmark tier, maximum-line category, or runtime-limit parameter. Although the number of cases and the value of k may help distinguish some benchmarks, they do not always uniquely determine the runtime tier. For example, benchmarks with the same number of cases and buckets may have either a 10M-line or 100M-line limit and therefore different runtime limits.

Could you please clarify the following?

1. Will the evaluation environment provide any explicit information indicating which benchmark or runtime tier is being evaluated?
2. Will the applicable maximum runtime be passed to the program through a command-line argument, environment variable, input metadata, or another mechanism?
3. If no such information is provided, should contestants design their programs under the most conservative 30-second runtime constraint for every benchmark?
4. Is the runtime limit enforced externally by the evaluation system, meaning that the submitted program does not need to determine the limit itself?

This information would help us design an appropriate adaptive strategy for processing large log files without risking timeout on smaller-runtime benchmarks.

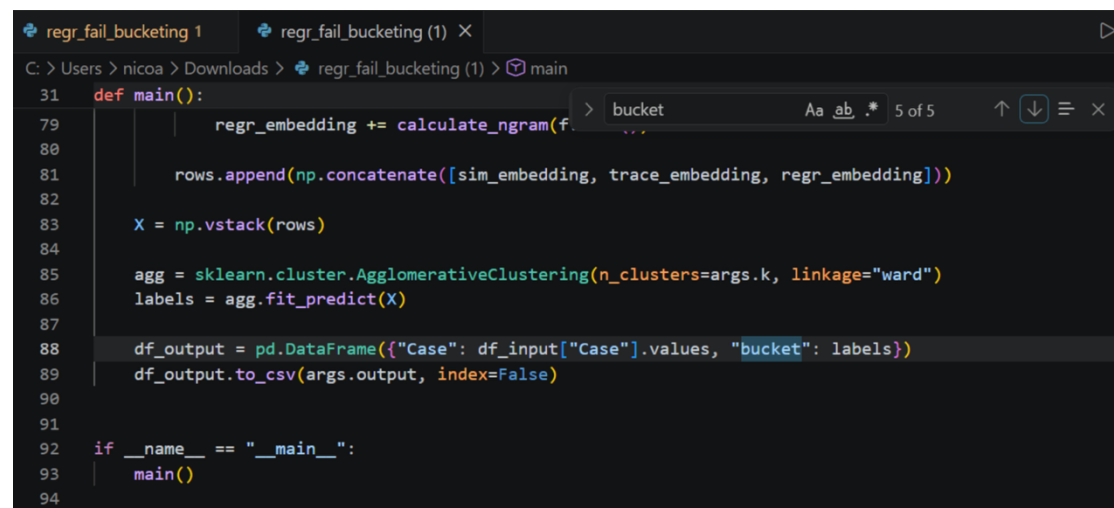
A20. The Topic Chair will use a script to run the program from each contestant. The runtime constraint and other benchmark info are only put on the outer script. The contestant should design the program according to Sec. 3.2. Problem Statement.

Q21. I would like to ask for clarification about the problem size of each benchmark of Problem B. Specifically, in the benchmark table, does "Max Lines" refer to the maximum number of lines in each individual log file, the total number of lines across the three log files of one case, or the total number of lines across all log files in the whole benchmark?

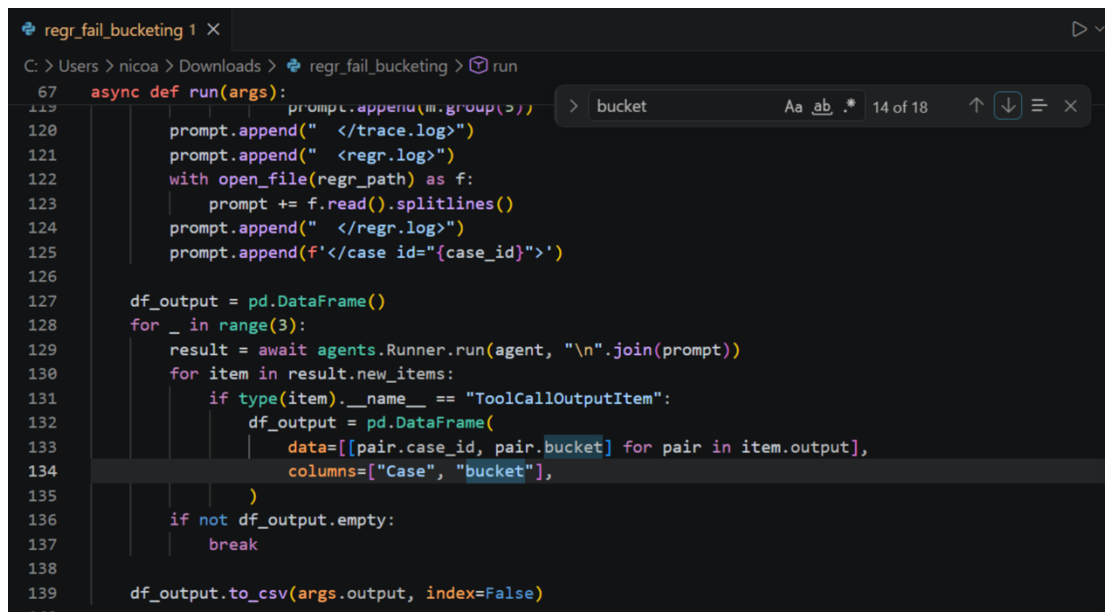
This clarification would help us better estimate the input scale and design our preprocessing flow.

A21. "Max Lines" refers to the total number of lines in three logs. Please notice that it is always dominated by sim.log while the other two are small.

Q22. As shown in the images below, the two provided sample code used the format "Case, bucket". Please confirm which format we should use, thank you very much.



```
regr_fail_bucketing 1  regr_fail_bucketing (1) X
C: > Users > nicoa > Downloads > regr_fail_bucketing (1) > main
31 def main():
79     regr_embedding += calculate_ngram(f
80
81     rows.append(np.concatenate([sim_embedding, trace_embedding, regr_embedding]))
82
83     X = np.vstack(rows)
84
85     agg = sklearn.cluster.AgglomerativeClustering(n_clusters=args.k, linkage="ward")
86     labels = agg.fit_predict(X)
87
88     df_output = pd.DataFrame({"Case": df_input["Case"].values, "bucket": labels})
89     df_output.to_csv(args.output, index=False)
90
91
92 if __name__ == "__main__":
93     main()
94
```



```
67  async def run(args):
119      prompt.append(m.group(1))
120      prompt.append("</trace.log>")
121      prompt.append("<regr.log>")
122      with open_file(regr_path) as f:
123          prompt += f.read().splitlines()
124      prompt.append("</regr.log>")
125      prompt.append(f'</case id="{case_id}">')
126
127      df_output = pd.DataFrame()
128      for _ in range(3):
129          result = await agents.Runner.run(agent, "\n".join(prompt))
130          for item in result.new_items:
131              if type(item).__name__ == "ToolCallOutputItem":
132                  df_output = pd.DataFrame(
133                      data=[[pair.case_id, pair.bucket] for pair in item.output],
134                      columns=["Case", "bucket"],
135                  )
136              if not df_output.empty:
137                  break
138
139      df_output.to_csv(args.output, index=False)
```

A22. The evaluation is case insensitive. Please avoid over-relying on the sample programs; they are just examples to demonstrate what the submitted program looks like. They will still fail on some benchmark sets.

Q23. While implementing and testing our bucketing engine, we noticed a minor inconsistency regarding the capitalization of the output CSV header in the official document (B_20260601.pdf).

According to Section 2.2 ("Format of Output Data"), the output table should contain two columns: Case (integer case ID) and bucket (bucket label), where the second column name is written in lowercase.

However, in Figure 2 ("Problem formulation: input table and output table"), the corresponding output table labels the second column as Bucket with a capitalized "B". To avoid any potential parsing issues during evaluation, we would greatly appreciate clarification on the following question:

Q1: Which exact column header is expected by the evaluation script for the second column? Should it be bucket or Bucket?

A23. The evaluation is case insensitive.

Q24. According to the official scoring formula:

True Positive Rate (TPR) = $TP / P = TP / (TP + FN)$

True Negative Rate (TNR) = $TN / N = TN / (TN + FP)$

Balanced Accuracy = $(TPR + TNR) / 2$

For both Set 1 and Set 2, the Balanced Accuracy scores announced in the Beta Test results are approximately 0.1 lower than the scores we calculated ourselves.

Could you please clarify whether we may have misunderstood how TP, TN, FP, and FN are defined or calculated? Alternatively, is there another part of the evaluation procedure that we may have overlooked?

According to the Q&A, Set 1 and Set 2 are public datasets and are evaluated in the same way as in the Beta Test. Therefore, we would appreciate further clarification on how the official scores for these two sets were calculated.

A24. Our Balanced Accuracy is computed pairwise, over all $C(N,2)$ pairs of cases (same-bug pairs vs. different-bug pairs), not a per-case, one-vs-rest classification.

TP = same-bug pairs you kept together

FP = different-bug pairs you merged together.

This is exactly the definition in the "Figure-of-Merit" section of the problem spec.

Q25. As we prepare for the Final submission, we would appreciate clarification on the following questions regarding the benchmark data and evaluation setup.

Q1. Regarding Q16, could you please clarify what “the same 10 benchmark sets” means for the Final evaluation? Please indicate which of the following is the closest description:

- A. Final uses byte-identical input files and log contents, with the same Case IDs, file paths, and row order.
- B. Log contents and Case IDs remain unchanged, but row order or file paths may change.
- C. The same cases and log contents are used, but Case IDs, row order, or file paths may all change.
- D. Cases or log contents may be regenerated using different seeds or simulation runs while representing the same underlying benchmark sets.

Q2. The Beta reports provide the Balanced Accuracy for each benchmark. Would it be possible to additionally provide the corresponding pairwise TPR and TNR for each benchmark in each contestant’s own Beta report, or privately to each team?

These aggregate values would not disclose any case-level labels, but would help contestants understand whether the dominant errors are false splits or false merges.

Q3. For each case, are `regr.log`, `sim.log.gz`, and `trace.log.gz` guaranteed to originate from the same simulation run and seed?

In addition, where applicable, can contestants assume that the primary failure or first mismatch reported in `regr.log` has a corresponding event or instruction somewhere in the associated simulation and/or trace logs, even if the exact indices and formatting differ?

Q4. Without disclosing any case-to-bucket mapping, would it be possible to provide aggregate golden-bucket size statistics for each benchmark, such as:

- the number of singleton buckets;
- the minimum, median, and maximum number of cases per bucket; and
- whether the bucket-size distribution is approximately balanced or highly imbalanced?

Only aggregate statistics are requested; no case-level labels are needed.

Q5. Within a benchmark, may cases caused by the same injected bug come from different regression tests or exhibit substantially different failure mechanisms—for example, an instruction mismatch in one case and a timeout, assertion, signature failure, or UVM error in another?

Alternatively, are cases belonging to the same bug generally generated from the same test and failure type, with differences mainly caused by seeds or runtime variation?

Q6. Benchmark pairs 3/7, 4/8, 5/9, and 6/10 have the same published maximum case count and bucket count, but different Max Lines limits. Are the two benchmarks in each pair independent bug/case populations, or do they use the same or related bug populations with different log sizes or simulation lengths?

We are only asking about the benchmark construction relationship, not the identities of any injected bugs.

A25. The benchmark sets for alpha/beta/final are byte-identical. The only difference is the timeout threshold to evaluate potential score. To help contestants optimize bucketing algorithms, we have included TPR/TNR for each benchmark set. The three logs per case are generated from one simulation, while different cases are generated by different bugs, tests, and seeds. Please be aware the number of failure cases per bug is imbalanced, and the error signature is both dependent on bug and test. Different benchmark sets are designed to test scalability and robustness across varying log sizes, case counts, and bug diversity; as with case-to-bug mappings, we don't disclose the internal construction relationships between specific benchmark sets.

Q26. May contextual or process-related information introduced during benchmark generation be used for bucketing?

A26. Submitted program should cluster failures according to functional features in logs. Please refer to the new section 3.7. Compliance and Resilience Checks